

Task 06 - Part 01: Question Answers

C# Fundamentals - Structs, Classes, Access Modifiers & Encapsulation

Q1. Why can't a struct inherit from another struct or class in C#?

Structs in C# are **sealed by design** - the language simply does not allow struct-to-struct or struct-to-class inheritance. This is intentional: structs are meant to be lightweight, simple value types that are copied by value, and supporting inheritance (with polymorphism, virtual methods, and reference-based dispatch) would work against that value-type nature and add overhead/complexity that goes against the purpose of a struct. The one exception is that every struct implicitly inherits from `System.ValueType` (which itself inherits from `System.Object`), and a struct **can implement interfaces** - it just cannot inherit from another struct or class.

Q2. How do access modifiers impact the scope and visibility of a class member?

Access modifiers control **which parts of the code are allowed to see and use a given class member** (field, property, or method). **private** restricts access to only within the same class - nothing outside it, not even a derived class, can see it. **internal** allows access from anywhere within the same assembly (project), but not from other assemblies. **public** allows access from anywhere, including other assemblies that reference this one. Choosing the right modifier is central to encapsulation: it determines how much of a class's internal implementation is exposed versus hidden, which directly affects how safely and predictably other parts of a program can interact with that class.

Q3. Why is encapsulation critical in software design?

Encapsulation bundles an object's data together with the methods that operate on it, while hiding the internal implementation details (typically via private fields) and exposing only a controlled public interface (via properties and methods). This matters because it **protects an object's internal state from being changed in invalid or unexpected ways** from outside code - for example, a `Salary` property can validate that a new value isn't negative before accepting it, something a public field could never enforce. It also makes code easier to maintain and refactor, since the internal implementation can change freely as long as the public interface stays the same, and it reduces the surface area of dependencies between different parts of a program.

Q4. What is a constructor in a struct?

A constructor in a struct is a special method, matching the struct's name, that runs when a new instance of the struct is created, and is responsible for initializing its fields. Structs always have an implicit **parameterless default constructor** available (which sets every field to its default value), but a struct can also define one or more **parameterized constructors** to initialize its fields with specific values at creation time - this is how constructor overloading is achieved for a struct, exactly as it is for a class. One key rule specific to structs is that every field must be definitely assigned by the end of a custom constructor before the constructor returns.

Q5. How does overriding methods like ToString() improve code readability?

By default, calling ToString() on a custom type just prints its fully-qualified type name (e.g., "Task06.Point"), which isn't useful for understanding the actual data an instance holds. **Overriding ToString()** lets a type define exactly how it should be represented as readable text - such as printing a Point as "(3, 5)" instead of its type name. This makes debugging, logging, and simple console output significantly clearer, since Console.WriteLine(someObject) (and string interpolation) automatically calls ToString() under the hood, so a well-designed override means the object's meaningful state is visible at a glance, wherever it's printed, without extra formatting code at every call site.

Q6. How does memory allocation differ for structs and classes in C#?

A **struct** is a value type: when declared as a local variable or passed to a method, its data is typically allocated directly on the **stack** (or inline within whatever object contains it), and assigning or passing it **copies the entire value** - the original and the copy are completely independent afterward. A **class** is a reference type: the object's actual data is allocated on the **heap**, and the variable itself just holds a reference (address) to that memory; assigning or passing it copies only the reference, so both variables end up pointing to the **same** underlying object, and changes made through one are visible through the other. This is exactly why passing a struct to a method leaves the original untouched by default, while passing a class instance lets the method modify the same object the caller is holding.